CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação
Prof. Marco André Abud Kappel

Aula 9 – *Avaliação de Modelos de Classificação*

Avaliação de Modelos de Classificação

- **Introdução**
  - Até aqui, utilizamos a **taxa de acerto** do algoritmo como o critério de validação para o mesmo.
  - Em problemas reais, o fato do algoritmo ser capaz de obter **bons resultados** em uma base de testes não é o suficiente para dizer que ele é melhor que outros.
  - Outros **fatores** podem influenciar na escolha da técnica de aprendizado de máquina:
    - Fatores importantes:
      - Verdadeiro positivo e falso positivo
      - Matriz de Confusão
      - Validação cruzada

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Real	Risco de Crédito		
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.
  - As **linhas** representam as classes às quais os dados pertencem.

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta impo um algoritmo.

Previsão

Real

	Risco de Crédito		
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.
  - As **linhas** representam as classes às quais os dados pertencem.
  - As **colunas** mostram a previsão que a técnica forneceu.

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.
  - As **linh**
  - As **colu**

A soma de cada linha mostra o total de registros da respectiva classe na base de dados.

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.
  - As linh
  - As colu

A soma da diagonal mostra o total de acertos que a técnica alcançou.
Neste caso: $28 + 32 + 25 = 85$

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.
  - As linh
  - As colu

A soma dos outros elementos mostra o total de erros.
Neste caso: $6 + 5 + 8 + 7 + 3 + 2 = 31$

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

  - Tanto nas linhas quanto as colunas mostram as classes do problema.

O total de registros na base é a soma de acertos mais erros:

$$85 + 31 = 116$$

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Ferramenta importante para avaliar os erros e acertos de um algoritmo.

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

Para calcular o percentual de acerto (acurácia), basta fazer:

$$\frac{85}{116} \cdot 100 = 73{,}27\ \%$$

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Este é o cálculo do “accuracy_score”, que temos usado até aqui.
  - Como saber se esta taxa é **satisfatória**?
  - Quatro formas comuns:
    1. **Análise de contexto**: se a taxa de acerto envolve decisões vitais e sem margem de erro, por exemplo, a taxa precisa se aproximar ao máximo de 100%.
    2. **Número de classes**: em um problema com duas classes possíveis, a chance de acertar aleatoriamente uma classificação é de 50%. Um algoritmo SEMPRE precisa ser melhor que isso (classificador trivial).
    3. **Comparação com outros resultados da literatura**.

Avaliação de Modelos de Classificação

- **Matriz de Confusão**
  - Dependendo da aplicação em que estamos trabalhando, pode ser interessante fazer outras análises dos dados da **matriz de confusão**.
  - Podemos ver a matriz de confusão da seguinte maneira:

Verdadeiro Positivo	Falso Negativo
Falso Positivo	Verdadeiro Negativo

  - Um **erro** de cada tipo pode ter **diferentes importâncias** em problemas.

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 1**

	Doente	Não doente
Doente	Verdadeiro Positivo	Falso Negativo
Não doente	Falso Positivo	Verdadeiro Negativo

  - Suponha que o problema envolve a **classificação** de pacientes que podem **possuir uma doença** grave ou não.
  - O nosso algoritmo deve ser capaz de detectar **suspeitas da doença**, por exemplo, para que **novos exames** sejam feitos.
  - Neste caso, qual é o **tipo de erro** que tem mais efeitos negativos?

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 1**

	Doente	Não doente
Doente	Verdadeiro Positivo	Falso Negativo
Não doente	Falso Positivo	Verdadeiro Negativo

  - O **falso negativo**!
  - Se um paciente estiver doente e não fizer novos exames, sua situação ficará mais **grave**.
  - Por outro lado, os resultados do tipo "**falso positivo**" não são erros graves, pois apenas ocasionarão exames em pacientes que não possuem a doença.

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 1**

	Doente	Não doente
Doente	Verdadeiro Positivo	Falso Negativo
Não doente	Falso Positivo	Verdadeiro Negativo

  - Nestes casos, as melhores técnicas serão aquelas que apresentam a maior taxa de "**Verdadeiros Positivos**" e menor taxa de "**Falsos Negativos**".
  - Assim, se o algoritmo errar, será julgando que alguém possui a doença, quando não possui.

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 2**

	É Spam	Não é Spam
É Spam	Verdadeiro Positivo	Falso Negativo
Não é Spam	Falso Positivo	Verdadeiro Negativo

  - Suponha que o problema envolve a **classificação** de mensagens como Spam ou não.
  - O nosso algoritmo deve ser capaz de detectar **suspeitas de Spam**, por exemplo, para **retirar da caixa de entrada**.
  - Neste caso, qual é o **tipo de erro** que tem mais efeitos negativos?

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 2**

	É Spam	Não é Spam
É Spam	Verdadeiro Positivo	Falso Negativo
Não é Spam	Falso Positivo	Verdadeiro Negativo

  - O **falso positivo**!
  - Se uma mensagem comum for classificada como Spam erroneamente, ela será **retirada da caixa de entrada** e o usuário pode não vê-la.
  - Por outro lado, os resultados do tipo "**falso negativo**" vai apenas fazer com que o usuário exclua a mensagem manualmente.

Avaliação de Modelos de Classificação

- **Matriz de Confusão: Exemplo 2**

	É Spam	Não é Spam
É Spam	Verdadeiro Positivo	Falso Negativo
Não é Spam	Falso Positivo	Verdadeiro Negativo

  - Nestes casos, as melhores técnicas serão aquelas que apresentam a maior taxa de "**Verdadeiros Negativos**" e menor taxa de "**Falsos Positivos**".
  - Assim, se o algoritmo errar, o usuário não perderá nenhuma mensagem importante.

Avaliação de Modelos de Classificação

- **Métricas de avaliação: *Acurácia***
  - Diz quanto o modelo acertou das previsões possíveis.
  - Basicamente, é a razão entre o somatório das previsões corretas (verdadeiros positivos com verdadeiros negativos) sobre o somatório das previsões.

$$Acurácia = \frac{VP + VN}{VP + FP + VN + FN} = \frac{predições\ corretas}{todas\ as\ predições}$$

  - Em geral, é a **métrica básica** para avaliar o **resultado geral** de um problema de classificação.

Avaliação de Modelos de Classificação

VP	FN
FP	VN

- **Métricas de avaliação: *Recall***
  - Para casos em que a previsão de uma das classes é **mais importante que a das demais**, a acurácia pode não ser a melhor métrica.
  - Uma métrica mais eficiente pode ser a chamada “recall”:

$$Recall = \frac{Verdadeiros\ Positivos}{Verdadeiros\ Positivos + Falsos\ Negativos}$$

  - Ela mede quantos casos de **uma classe** foram capturados por **previsões corretas**.
  - Ou seja, mostra quantas previsões de uma classe foram corretas, dentro do **total de valores** que a classe possui.

Avaliação de Modelos de Classificação

VP	FN
FP	VN

- **Métricas de avaliação: *Precisão***
  - Tem como objetivo responder à pergunta: "Qual a **proporção** de identificações positivas que foram realmente corretas?"
  - Quando a precisão é alta para uma classe, a probabilidade da classe ter sido corretamente identificada também é alta.

$$Precisão = \frac{Verdadeiros\ Positivos}{Verdadeiros\ Positivos + Falsos\ Positivos}$$

  - Ou seja, do total de previsões que o modelo fez **indicando positivo para uma classe específica**, quantas foram corretas?

Avaliação de Modelos de Classificação

- **Métricas de avaliação: *F1-score***
  - Faz um **balanço** entre a **precisão** e o **recall** do modelo.
  - Calcula a **média harmônica** entre a precisão e o recall, ou seja, possui valores melhores quando as duas métricas estão bem balanceadas.

$$F1\text{–}score = 2 \times \frac{precisão \times recall}{precisão + recall}$$

  - O valor máximo é igual a 1, indicando que **tanto a precisão quanto o recall** estão bons para determinada classe.

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = \frac{VP + VN}{VP + FP + VN + FN}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = \frac{28 + 32 + 25}{28 + 7 + 3 + 32 + 6 + 2 + 25 + 5 + 8}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = \frac{28 + 32 + 25}{28 + 7 + 3 + 32 + 6 + 2 + 25 + 5 + 8} = \frac{85}{116} = 0{,}7327$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = \frac{VP}{VP + FN}$$

$$Recall(Moderado) = \frac{VP}{VP + FN}$$

$$Recall(Baixo) = \frac{VP}{VP + FN}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = \frac{28}{28 + 7 + 3}$$

$$Recall(Moderado) = \frac{32}{32 + 6 + 2}$$

$$Recall(Baixo) = \frac{25}{25 + 5 + 8}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = \frac{28}{28+7+3} = \frac{28}{38} = 0{,}7368$$

$$Recall(Moderado) = \frac{32}{32+6+2} = \frac{32}{40} = 0{,}8$$

$$Recall(Baixo) = \frac{25}{25+5+8} = \frac{25}{38} = 0{,}6578$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = 0{,}7368$$

$$Recall(Moderado) = 0{,}8$$

$$Recall(Baixo) = 0{,}6578$$

$$Precisão(Alto) = \frac{VP}{VP + FP}$$

$$Precisão(Moderado) = \frac{VP}{VP + FP}$$

$$Precisão(Baixo) = \frac{VP}{VP + FP}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = 0{,}7368$$

$$Recall(Moderado) = 0{,}8$$

$$Recall(Baixo) = 0{,}6578$$

$$Precisão(Alto) = \frac{28}{28 + 6 + 5}$$

$$Precisão(Moderado) = \frac{32}{32 + 7 + 8}$$

$$Precisão(Baixo) = \frac{25}{25 + 3 + 2} = \frac{25}{30}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$$Recall(Alto) = 0{,}7368$$

$$Recall(Moderado) = 0{,}8$$

$$Recall(Baixo) = 0{,}6578$$

$$Precisão(Alto) = \frac{28}{28+6+5} = \frac{28}{39} = 0{,}7179$$

$$Precisão(Moderado) = \frac{32}{32+7+8} = \frac{32}{47} = 0{,}681$$

$$Precisão(Baixo) = \frac{25}{25+3+2} = \frac{25}{30} = 0{,}833$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$
$$Recall(Alto) = 0{,}7368$$
$$Recall(Moderado) = 0{,}8$$
$$Recall(Baixo) = 0{,}6578$$
$$Precisão(Alto) = 0{,}7179$$
$$Precisão(Moderado) = 0{,}681$$
$$Precisão(Baixo) = 0{,}833$$

$$F1\text{–}score(Alto) = 2 \times \frac{precisão \times recall}{precisão + recall}$$

$$F1\text{–}score(Moderado) = 2 \times \frac{precisão \times recall}{precisão + recall}$$

$$F1\text{–}score(Baixo) = 2 \times \frac{precisão \times recall}{precisão + recall}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$
$$Recall(Alto) = 0{,}7368$$
$$Recall(Moderado) = 0{,}8$$
$$Recall(Baixo) = 0{,}6578$$
$$Precisão(Alto) = 0{,}7179$$
$$Precisão(Moderado) = 0{,}681$$
$$Precisão(Baixo) = 0{,}833$$

$$F1\text{–}score(Alto) = 2 \times \frac{0{,}7179 \times 0{,}7368}{0{,}7179 + 0{,}7368}$$

$$F1\text{–}score(Moderado) = 2 \times \frac{0{,}681 \times 0{,}8}{0{,}681 + 0{,}8}$$

$$F1\text{–}score(Baixo) = 2 \times \frac{0{,}833 \times 0{,}6578}{0{,}833 + 0{,}6578}$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$
$$Recall(Alto) = 0{,}7368$$
$$Recall(Moderado) = 0{,}8$$
$$Recall(Baixo) = 0{,}6578$$
$$Precisão(Alto) = 0{,}7179$$
$$Precisão(Moderado) = 0{,}681$$
$$Precisão(Baixo) = 0{,}833$$

$$F1\text{–}score(Alto) = 2 \times \frac{0{,}7179 \times 0{,}7368}{0{,}7179 + 0{,}7368} = \frac{1{,}0579}{1{,}4547} = 0{,}73$$

$$F1\text{–}score(Moderado) = 2 \times \frac{0{,}681 \times 0{,}8}{0{,}681 + 0{,}8} = \frac{1{,}089}{1{,}481} = 0{,}74$$

$$F1\text{–}score(Baixo) = 2 \times \frac{0{,}833 \times 0{,}6578}{0{,}833 + 0{,}6578} = \frac{1{,}096}{1{,}491} = 0{,}74$$

Avaliação de Modelos de Classificação

- **Exemplo**
  - No problema de risco de crédito, temos:

Risco de Crédito			
	Alto	Moderado	Baixo
Alto	28	7	3
Moderado	6	32	2
Baixo	5	8	25

$$Acurácia = 0{,}7327$$

$Recall(Alto) = 0{,}7368$ $\quad$ $Recall(Moderado) = 0{,}8$ $\quad$ $Recall(Baixo) = 0{,}6578$

$Precisão(Alto) = 0{,}7179$ $\quad$ $Precisão(Moderado) = 0{,}681$ $\quad$ $Precisão(Baixo) = 0{,}833$

$F1\text{–}score(Alto) = 0{,}73$ $\quad$ $F1\text{–}score(Moderado) = 0{,}74$ $\quad$ $F1\text{–}score(Baixo) = 0{,}74$

- **Construindo o script**

  – No script, podemos calcular essas métricas incluindo o seguinte código após o treinamento do classificador:

```
# Avaliação de métricas
from sklearn.metrics import classification_report
valores_classe = np.unique(base[classe.columns].values)
metricas_resumo = classification_report(classe_teste, previsoes, target_names=valores_classe)
```

  – Por exemplo, se executarmos estas linhas após o treinamento do SVM utilizando nossa base de crédito do banco alemão, teremos o seguinte resultado para as métricas:

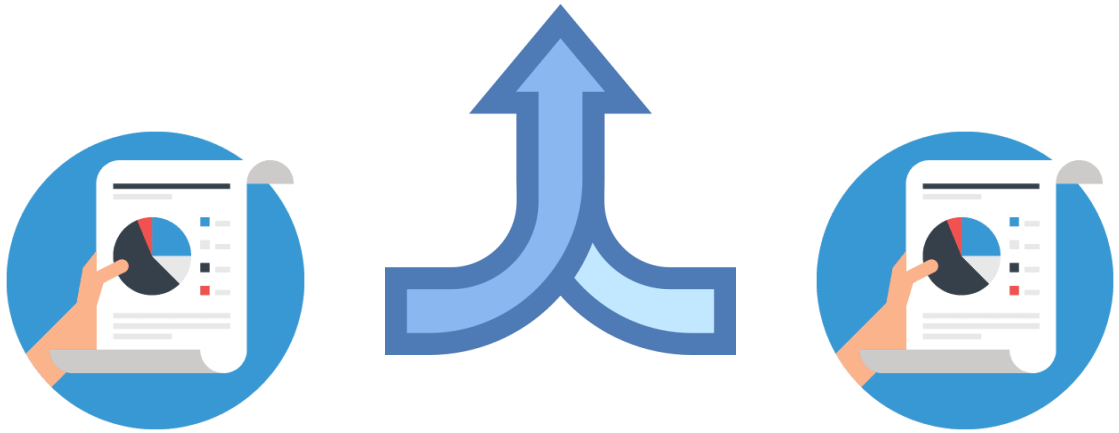
Editor de texto - metricas_resumo

	precision	recall	f1-score	support
bad	0.78	0.09	0.17	74
good	0.72	0.99	0.83	176
accuracy			0.72	250
macro avg	0.75	0.54	0.50	250
weighted avg	0.74	0.72	0.64	250

Salvar e Fechar Fechar

Avaliação de Modelos de Classificação

- **Validação Cruzada (Cross-validation)**
  - É um **método estatístico** de avaliar a capacidade de **generalização** de um método.
  - É mais **estável** e **confiável** que usar apenas a divisão em base de treinamento e testes.
  - Na validação cruzada, os dados são divididos **repetidas vezes** e diferentes modelos são treinados.



Avaliação de Modelos de Classificação

- **Validação Cruzada (Cross-validation)**
  - A versão mais conhecida e utilizada da validação cruzada é chamada "*K-Fold Cross-validation*".
  - Nesta abordagem a base de dados original é dividida em K "dobras".
  - Depois, o **treinamento** é feito **K vezes**:

Execução 1	Teste	Treinamento	Treinamento	Treinamento	Treinamento
Execução 2	Treinamento	Teste	Treinamento	Treinamento	Treinamento
Execução 3	Treinamento	Treinamento	Teste	Treinamento	Treinamento
Execução 4	Treinamento	Treinamento	Treinamento	Teste	Treinamento
Execução 5	Treinamento	Treinamento	Treinamento	Treinamento	Teste
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

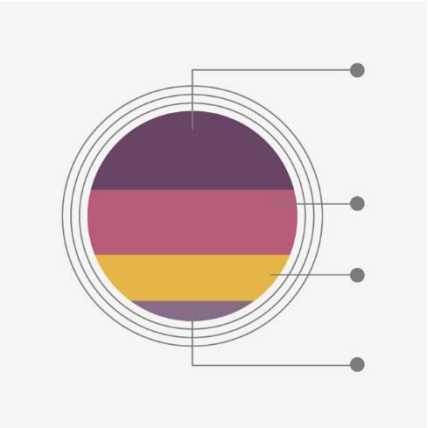
Avaliação de Modelos de Classificação

- **Validação Cruzada (Cross-validation)**
  - Em cada rodada, uma **parte diferente** da base é utilizada como base de **testes**, e as restantes são usadas no **treinamento**.
  - No final, teremos **K** resultados diferentes.
  - O resultado final será a **média simples** das métricas calculadas.

Execução 1	Teste	Treinamento	Treinamento	Treinamento	Treinamento
Execução 2	Treinamento	Teste	Treinamento	Treinamento	Treinamento
Execução 3	Treinamento	Treinamento	Teste	Treinamento	Treinamento
Execução 4	Treinamento	Treinamento	Treinamento	Teste	Treinamento
Execução 5	Treinamento	Treinamento	Treinamento	Treinamento	Teste
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

Avaliação de Modelos de Classificação

- **Validação Cruzada (Cross-validation)**
  - Para impedir que aconteça uma separação injusta, ou seja, que o **balanceamento das classes** seja afetado pela separação em folds, existe uma **variação** da validação cruzada chamada: “Stratified k-Fold Cross-Validation”.
  - A diferença é que, neste caso, os dados não serão simplesmente divididos, mas a divisão será feita de maneira a deixar a **mesma proporção** de classes em **cada partição**.



- **Validação Cruzada (Cross-validation)**
  - Por exemplo, se os dados estiverem com a seguinte disposição:

Previsores	Classe
...	1
...	1
...	1
...	2
...	2
...	2

K-Fold

Stratified K-Fold

Fold 1

Previsores	Classe
...	1
...	1

Fold 1

Previsores	Classe
...	1
...	2

Fold 2

Previsores	Classe
...	1
...	2

Fold 2

Previsores	Classe
...	1
...	2

Fold 3

Previsores	Classe
...	2
...	2

Fold 3

Previsores	Classe
...	1
...	2

- **Construindo o script**
  - Primeiramente, devemos identificar quais foram os melhores resultados que obtivemos.

Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão
Classe Majoritária	-	-	0,704	0 74 0 176
KNN	K = 32, Distância Euclidiana	Padronização, OneHotEncoder	0,716	9 65 6 170
Naive Bayes	Gaussiana	Padronização	0,712	4 70 2 174
Árvores de Decisão	Altura máxima = 2, Métrica=Entropia	-	0,708	4 70 3 173
Random Forests	Altura máxima = 3, Métrica=Gini, Nº de árvores=11, Nº de características=5,	-	0,72	10 64 6 170
SVM	Kernel = ‘sigmoid’ Gamma = ‘auto’ C = 0.9	Padronização, OneHotEncoder	0.724	7 67 2 174
Redes Neurais	max_iter=1000 tol = 0.000001 solver = 'sgd' hidden_layer_sizes=[10] activation='relu'	Padronização	0.736	14 60 6 170

- **Construindo o script**
  - Primeiramente, devemos identificar quais foram os melhores resultados que obtivemos.

Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão	
Classe Majoritária	-	-	0,704	0 0	74 176
KNN	K = 32, Distância Euclidiana	Padronização, OneHotEncoder	0,716	9 6	65 170
Naive Bayes	Gaussiana	Padronização	0,712	4 2	70 174
Árvores de Decisão	Altura máxima = 2, Métrica=Entropia	-	0,708	4 3	70 173
Random Forests	Altura máxima = 3, Métrica=Gini, Nº de árvores=11, Nº de características=5,	-	0,72	10 6	64 170
SVM	Kernel = 'sigmoid' Gamma = 'auto' C = 0.9	Padronização, OneHotEncoder	0.724	7 2	67 174
Redes Neurais	max_iter=1000 tol = 0.000001 solver = 'sgd' hidden_layer_sizes=[10] activation='relu'	Padronização	0.736	14 6	60 170

Vamos fazer a validação cruzada para estas configurações de técnicas.

- **Construindo o script**

  – O script para fazer a **validação cruzada** fica assim, por exemplo, para a utilização das **Redes Neurais**:

```
#  Não vou separar a base de dados em treinamento e teste

# Classificador
from sklearn.neural_network import MLPClassifier

# Divisão dos dados em folds
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, confusion_matrix, precision_recall_fscore_support
import numpy as np

kfold = StratifiedKFold(n_splits = 5, shuffle = True, random_state = 1)
acuracias = []
matrizes = []
metricas = []
for indice_treinamento, indice_teste in kfold.split(previsores,
                                                    np.zeros(shape=(previsores.shape[0],1))):
    classificador = MLPClassifier(verbose = True,
                                  max_iter=1000,
                                  tol = 0.000001,
                                  solver = 'sgd',
                                  hidden_layer_sizes = [10],
                                  activation = 'relu',
                                  random_state = 1)
    classificador.fit(previsores[indice_treinamento], classe.iloc[indice_treinamento,0])

    previsoes = classificador.predict(previsores[indice_teste])
    acuracia = accuracy_score(classe.iloc[indice_teste,0], previsoes)
    metricas.append(precision_recall_fscore_support(classe.iloc[indice_teste,0], previsoes))
    matrizes.append(confusion_matrix(classe.iloc[indice_teste,0], previsoes))
    acuracias.append(acuracia)
```

- **Construindo o script**
  - O script para fazer a **validação cruzada** fica assim, por exemplo, para a utilização das **Redes Neurais**:

```
#  Não vou separar a base de dados em treinamento e teste

# Classificador
from sklearn.neural_network import MLPClassifier

# Divisão dos dados em folds
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, confusion_matrix, precision_
import numpy as np

kfold = StratifiedKFold(n_splits = 5, shuffle = True, random_state = 1)
acuracias = []
matrizes = []
metricas = []
for indice_treinamento, indice_teste in kfold.split(previsores,
                                                    np.zeros(shape=(previsores.shape[0],1))):
    classificador = MLPClassifier(verbose = True,
                                  max_iter=1000,
                                  tol = 0.000001,
                                  solver = 'sgd',
                                  hidden_layer_sizes = [10],
                                  activation = 'relu',
                                  random_state = 1)
    classificador.fit(previsores[indice_treinamento], classe.iloc[indice_treinamento,0])

    previsoes = classificador.predict(previsores[indice_teste])
    acuracia = accuracy_score(classe.iloc[indice_teste,0], previsoes)
    metricas.append(precision_recall_fscore_support(classe.iloc[indice_teste,0], previsoes))
    matrizes.append(confusion_matrix(classe.iloc[indice_teste,0], previsoes))
    acuracias.append(acuracia)
```

Para usar outros classificadores, basta trocar nestes locais.

- **Construindo o script**

  – Depois, devemos calcular os resultados médios para as métricas:

```
######################### Resultado final #########################
# Matriz de confusão média
matriz_media = np.mean(matrizes, axis = 0)
matriz_desvio_padrao = np.std(matrizes, axis = 0)
# Métricas médias
acuracias = np.asarray(acuracias)
acuracia_final_media = acuracias.mean()
acuracia_final_desvio_padrao = acuracias.std()
metricas_medias = np.mean(metricas, axis = 0)
metricas_desvio_padrao = np.std(metricas, axis = 0)
#Obs: linhas: precisao, recall, f1score // colunas: classe
```

- **Construindo o script**

  – Depois, devemos calcular os resultados médios para as métricas:

```
######################## Resultado final ########################
# Matriz de confusão média
matriz_media = np.mean(matrizes, axis = 0)
matriz_desvio_padrao = np.std(matrizes, axis = 0)
# Métricas médias
acuracias = np.asarray(acuracias)
acuracia_final_media = acuracias.mean()
acuracia_final_desvio_padrao = acuracias.std()
metricas_medias = np.mean(metricas, axis = 0)
metricas_desvio_padrao = np.std(metricas, axis = 0)
#Obs: linhas: precisao, recall, f1score // colunas: classe
```

Os resultados podem variar dos obtidos antes, pois agora representam a média dos ajustes.

acuracia_final_desvio_padrao	float64	1	0.029597297173897467
acuracia_final_media	float64	1	0.7120000000000001
matriz_desvio_padrao	Array of float64	(2, 2)	[[1.95192213 3.1] [2.21133444 3.53411941]]
matriz_media	Array of float64	(2, 2)	[[5.3 24.7] [4.1 65.9]]
metricas_desvio_padrao	Array of float64	(4, 2)	[[0.17630666 0.03309263] [0.05739179 0.03015023] [0.07371221 0.02253
metricas_medias	Array of float64	(4, 2)	[[0.58156732 0.72744055] [0.17507812 0.94190474] [0.26358759 0

- **Construindo o script**

  – Depois, devemos calcular os resultados médios para as métricas:

```
#######################
# Matriz de confusão
matriz_media = np.mean
matriz_desvio_padrao =
# Métricas médias
acuracias = np.asarray
acuracia_final_media =
acuracia_final_desvio
metricas_medias = np.
metricas_desvio_padrao
#Obs: linhas: precisao
```

bad | good

	0	1
0	0.581567	0.727441
1	0.175078	0.941905
2	0.263588	0.820158
3	30	70

Precisão
Recall
F1-score

Formato | Redimensionar | Cor de fundo

Salvar e Fechar | Fechar

		1	0.029597297173897467
		1	0.7120000000000001
matriz_desvio_padrao	Array of float64	(2, 2)	[[1.95192213 3.1] [2.21133444 3.53411941]]
matriz_media	Array of float64	(2, 2)	[[5.3 24.7] [4.1 65.9]]
metricas_desvio_padrao	Array of float64	(4, 2)	[[0.17630666 0.03309263] [0.05739179 0.03015023] [0.07371221 0.02253
metricas_medias	Array of float64	(4, 2)	[[0.58156732 0.72744055] [0.17507812 0.94190474] [0.26358759 0

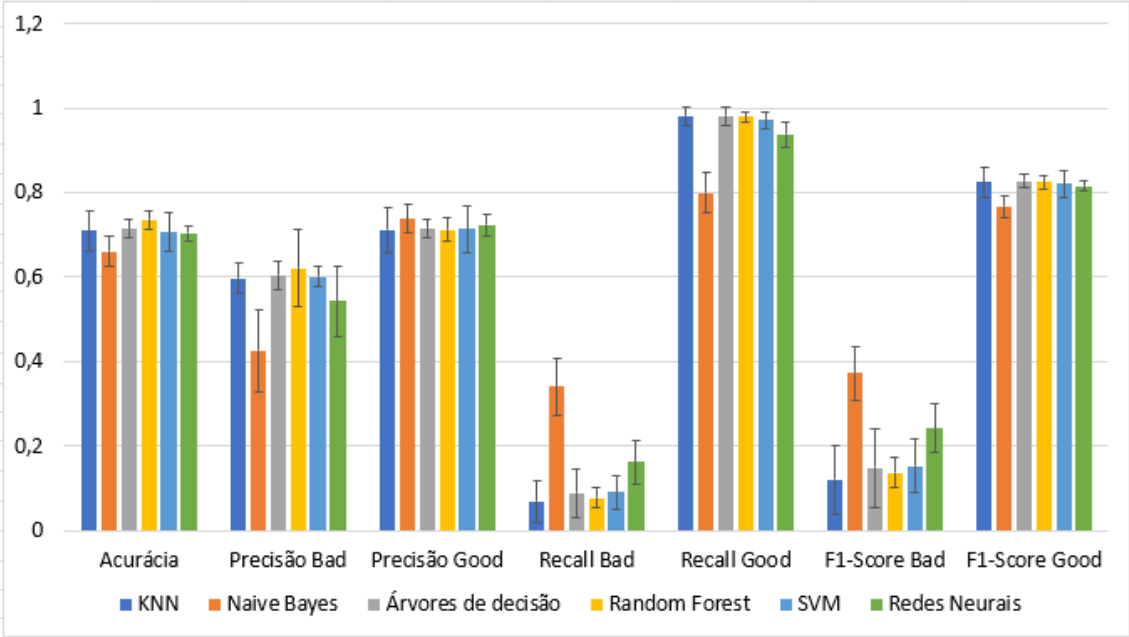
Devemos registrar os resultados em um novo arquivo.

- **Construindo o script**
  - O novo arquivo, contendo os resultados finais para as principais configurações testadas, deve ficar assim:

Dados de Crédito

	Médias						
Método	Acurácia	Precisão Bad	Precisão Good	Recall Bad	Recall Good	F1-Score Bad	F1-Score Good
KNN	0,709	0,596667	0,711944	0,0697235	0,981921	0,121315	0,824014
Naive Bayes	0,66	0,425456	0,738555	0,340177	0,799807	0,372949	0,766701
Árvores de decisão	0,714	0,604516	0,715618	0,0884074	0,981267	0,149052	0,827368
Random Forest	0,735	0,620433	0,712326	0,0782482	0,978829	0,13736	0,824177
SVM	0,706	0,600595	0,713667	0,0908306	0,970963	0,152352	0,821051
Redes Neurais	0,704	0,542809	0,723109	0,162293	0,937716	0,242059	0,81578

	Desvios						
	Acurácia	Precisão Ba	Precisão G	Recall Ba	Recall Good	F1-Score Ba	F1-Score G
KNN	0,047212	0,0364676	0,0535152	0,049397	0,022225	0,0809956	0,033812
Naive Bayes	0,035777	0,0968695	0,0338438	0,065996	0,0471745	0,0629073	0,025219
Árvores de decisão	0,022226	0,0348272	0,0219874	0,057481	0,0209278	0,093177	0,016724
Random Forest	0,023367	0,0905755	0,026363	0,022019	0,0102502	0,0348181	0,015576
SVM	0,046087	0,0244623	0,0556479	0,040381	0,0197451	0,0639453	0,032282
Redes Neurais	0,017436	0,0830235	0,026834	0,051092	0,0289025	0,0566146	0,012733



Matriz média	
KNN	
1	59
0,6	139,4

Matriz média	
Naive Bayes	
20,2	39,8
28,2	111,8

Matriz média	
Árvores de decisão	
5,4	54,6
2,6	137,4

Matriz desvio	
KNN	
1,09545	4,14729
0,489898	4,71593

Matriz desvio	
Naive Bayes	
3,31059	6,61513
7,24983	4,26146

Matriz desvio	
Árvores de decisão	
3,55528	4,58694
2,87054	6,0531

Matriz média	
Random Forest	
4,6	55,4
3	137

Matriz média	
SVM	
4	56
4,4	135,6

Matriz média	
Redes Neurais	
9,6	50,4
8,8	131,2

Matriz desvio	
Random Forest	
1,0198	5,64269
1,54919	3,74166

Matriz desvio	
SVM	
1,89737	5,32917
2,24499	3,44093

Matriz desvio	
Redes Neurais	
2,57682	6,15142
4,26146	3,91918

FIM